

Informe técnico

Base de Datos Intermitente y Condición de Carrera

Práctica: Tolerancia a fallos
Sistema: Reservas de entradas en Kubernetes multi-nodo
Modalidad: Trabajo individual
Estudiante: _____
Fecha: julio de 2026

Fallo	Riesgo principal	Solución central
BD intermitente	resultado ambiguo y duplicados	idempotencia + transacción + outbox
Carrera	doble venta del último asiento	operación atómica + restricción única

Alcance y selección

Los cuatro fallos ejecutados son caída de Inventario, latencia de Pagos, sobrecarga del Gateway y caída de Notificaciones. Este informe analiza conectividad intermitente con PostgreSQL y condición de carrera por el último asiento. Ambos comprometen datos y requieren más que reiniciar pods.

1. Base de Datos Intermitente

1.1 Por qué ocurre

El flapping puede originarse en pérdida de paquetes, reinicio del endpoint, agotamiento del pool, NAT o failover. Una escritura tiene fases de envío, ejecución/commit y respuesta. Si la conexión se corta después del COMMIT pero antes de recibir confirmación, el cliente observa un resultado desconocido: reintentar ciegamente puede duplicar cobro o reserva.

CAP y consistencia

CAP no significa escoger siempre dos letras. Solo durante una partición existe tensión entre consistencia y disponibilidad. Para ventas se prioriza la consistencia de inventario y pagos: si no puede confirmarse el líder autoritativo se devuelve un error reintentable, no una venta potencialmente doble. Puede mantenerse disponibilidad parcial para consultas, pero no para descontar el último asiento.

Timeouts y retries

Los timeouts se ordenan: conexión menor que consulta, y consulta menor que el presupuesto HTTP. Un pool ilimitado convierte la partición en agotamiento. El retry solo es seguro para errores transitorios y operaciones idempotentes; validaciones y conflictos no se reintentan.

1.2 Solución de producción

PostgreSQL administrado u operador con réplica y backups; PgBouncer con pool acotado; clave de idempotencia única; transacción corta que guarda reserva y evento Outbox; retries con backoff/jitter para SQLSTATE transitorio; y métricas de pool, p95/p99, SQLSTATE, abortos y lag. Ante resultado ambiguo, se consulta la clave antes de repetir.

1.3 Pseudocódigo

```
comprar(cmd, key):
  previo = SELECT respuesta WHERE clave = key
  si existe: retornar previo
  repetir máximo 3 veces:
    BEGIN SERIALIZABLE
    INSERT solicitud(key, 'EN_PROCESO')
    reservar_asiento_atómico(cmd.seatId)
    INSERT reserva(...)
    INSERT outbox('ReservaCreada', payload)
    UPDATE solicitud SET estado='COMPLETA'
  COMMIT
  ante resultado ambiguo: consultar key en conexión nueva
```

1.4 Validación

Inyectar pérdida intermitente con Toxiproxy o NetworkPolicy. Verificar una sola fila por clave, errores 503 acotados, pool estable y recuperación al restablecer la ruta.

2. Condición de Carrera: último asiento

2.1 Por qué ocurre

Con “leer disponibilidad; luego descontar”, dos transacciones pueden leer disponible antes de que cualquiera escriba. Ambas confirman y existe doble venta. Réplicas de Kubernetes amplían la concurrencia; un mutex local no coordina pods y se pierde al reiniciar.

Aislamiento e invariante

Read Committed evita lecturas sucias, pero no garantiza que una decisión basada en lectura siga válida. El invariante es: por evento y asiento existe como máximo una reserva activa. Debe residir en el almacén autoritativo.

2.2 Solución de producción

Usar UPDATE condicional con RETURNING o SELECT FOR UPDATE, más una restricción única (event_id, seat_id). Los holds con expiración permiten pagar sin bloquear durante una llamada de red. El flujo saga crea el hold, cobra con idempotencia y confirma; si falla, libera o expira.

2.3 Pseudocódigo

```
crearHold(eventId, seatId, buyerId, key):
BEGIN
INSERT hold(..., expires_at=now()+5min)
ON CONFLICT(event_id, seat_id) DO NOTHING
si filas = 0: ROLLBACK; retornar 409
UPDATE seat SET status='HELD'
COMMIT; retornar 201

confirmarHold(holdId, paymentId):
UPDATE hold SET status='CONFIRMED'
WHERE id=? AND status='ACTIVE' AND expires_at>now()
UPDATE seat SET status='SOLD'; INSERT outbox(...); COMMIT
```

2.4 Prueba determinista

Inicializar un asiento y lanzar 50 clientes sincronizados. Resultado: exactamente un 201, cuarenta y nueve 409, una fila confirmada y ningún cobro duplicado. Repetir reiniciando un pod demuestra que el invariante reside en la base.

Conclusiones

Kubernetes recupera capacidad de cómputo; la idempotencia, las transacciones y las restricciones protegen la verdad del negocio. El fallo intermitente exige tratar resultados ambiguos; la carrera exige una operación atómica en el sistema autoritativo.

Referencias

PostgreSQL 16: Transaction Isolation y Explicit Locking. Kleppmann, Designing Data-Intensive Applications (2017). Richardson, Microservices Patterns (2018). Nygard, Release It!, 2.ª ed. (2018).

Nota: el repositorio recibido no incluía la tarea MLR previa; debe agregarse su referencia específica si corresponde, sin inventar una cita.